

Professor Orientador: Lucy Mari

Integrantes: Fabio Augusto, Marcos Almeida Daloia e André Seiji Shiroma

1. Estrutura Sequencial

A estrutura sequencial é utilizada no momento inicial de interação, onde o sistema coleta dados do estudante para personalizar a experiência de navegação.

Exemplo de Aplicação: Cadastro de Perfil

Entrada de dados (Sequencial)

```
nome_aluno = input("Digite o nome completo do aluno: ")
```

```
idade_aluno = int(input("Digite a idade do aluno: "))
```

```
turma_aluno = input("Digite a série/turma: ")
```

Processamento e Saída

```
exibicao_perfil = "Perfil criado: " + nome_aluno + " | Turma: " + turma_aluno
```

```
print(exibicao_perfil)
```

Resultado Aplicado:

- Entrada: "Marcos", "19", "ADS 2º Semestre"
 - Saída: "Perfil criado: André | Turma: ADS 2º Semestre"
-

3. Aplicação de Vetores

Os vetores foram utilizados para armazenar listas de jogos disponíveis na plataforma.

Exemplo de Aplicação: Lista de Jogos

```
jogos = ["Matemática Básica", "Português", "Ciências"]
```

Entrada de dados

- Lista de jogos cadastrados

Processamento

- Armazenamento dos dados em vetor

Saída

```
print(jogos)
```

Resultado Aplicado:

- Entrada: ["Matemática Básica", "Português", "Ciências"]
 - Saída: Exibição da lista de jogos
-

4. Aplicação de Matrizes

As matrizes foram utilizadas para armazenar o desempenho dos alunos nos jogos.

Exemplo de Aplicação: Notas dos Alunos

```
notas = [  
[8, 7],  
[9, 10]
```

3. Aplicação de Vetores

Os vetores foram utilizados para armazenar listas de jogos disponíveis na plataforma.

Exemplo de Aplicação: Lista de Jogos

```
jogos = ["Matemática Básica", "Português", "Ciências"]
```

Entrada de dados

- Lista de jogos cadastrados

Processamento

- Armazenamento dos dados em vetor

Saída

```
print(jogos)
```

Resultado Aplicado:

- Entrada: ["Matemática Básica", "Português", "Ciências"]
 - Saída: Exibição da lista de jogos
-

Entrada de dados

- Notas dos alunos

Processamento

- Organização em formato de tabela (matriz)

Saída

```
print(notas)
```

Resultado Aplicado:

- Entrada: [[8,7], [9,10]]
 - Saída: Exibição das notas
-

5. Aplicação da Modularização

A modularização foi utilizada para organizar o sistema em funções, facilitando a reutilização e manutenção do código.

Exemplo de Aplicação: Função de Perfil

```
def exibir_perfil(nome, turma):  
    return "Perfil: " + nome + " | Turma: " + turma
```

Entrada de dados

- Nome e turma do aluno

Processamento

- Organização dos dados por função

Saída

```
print(exibir_perfil("André", "ADS"))
```

Resultado Aplicado:

- ☐ Entrada: "André", "ADS"
 - ☐ Saída: "Perfil: André | Turma: A"
-

6. Observações Reflexivas do Grupo

Durante o desenvolvimento do projeto, o grupo encontrou dificuldades principalmente na utilização de matrizes, devido à organização dos dados em linhas e colunas.

- Com ajustes e testes, os resultados passaram a ser mais precisos e de acordo com o esperado. A modularização contribuiu para melhorar a estrutura do sistema, tornando o código mais organizado e fácil de entender.
 - Como melhoria futura, o grupo pretende implementar novas funcionalidades e melhorar a interface da plataforma, além de otimizar o tratamento de dados.
-